

```
print("Hello, World!")
```



LINGUAGEM DE PROGRAMAÇÃO PYTHON

AULA 08 - REVISÃO

ANTERIORMENTE

- **Compreender o conceito de debugging e sua importância.**
- **Identificar e diferenciar os três principais tipos de bugs.**
- **Aprender a decifrar mensagens de erro (Tracebacks).**
- **Utilizar ferramentas essenciais.**
- **Aplicar um fluxo de trabalho para resolver problemas.**
- **Try - Except**

PRÓXIMOS PASSOS

- **Revisão:**
 - **Características da Linguagem;**
 - **Operadores e Strings;**
 - **Estruturas Condicionais;**
 - **Laços de Repetição;**
 - **Funções;**
 - **Estruturas de dados;**
 - **Tratamento de Erros;**

Revisão

Está chegando o dia da prova!
Vamos revisar o conteúdo?



Histórico

Quem? Quando? Porque?

Criado por Guido van Rossum em dezembro de 1989, na Holanda, para tornar a programação mais fácil e intuitiva através da legibilidade do código. Nome em homenagem ao grupo de comédia britânico Monty Python!

Características.

Utiliza indentação ao invés de chaves ou palavras, tipagem dinâmica e forte.

Ambientes.

IDEs: Pycharm, VsCode.

Ambiente interativo: Jupyter e Colab.



PEP 08

- 1. Indentação:** 4 espaços por nível de indentação.
- 2. Tamanho das Linhas:** Limite as linhas de código a um máximo de 79 caracteres.
- 3. Linhas em Branco:** Separe funções e classes principais com duas linhas em branco.
- 4. Padrões de Nomenclatura (Naming Conventions):**
snake_case para variáveis e funções;
PascalCase para classes e;
UPPER_SNAKE_CASE para constantes.
- 5. Espaços em Operadores e Atribuições:** Espaço antes e depois de operadores matemáticos (+, -, *, /) e de atribuição (=). Não use espaços logo após parênteses, colchetes ou chaves.

Variáveis

- Python possui tipagem dinâmica: identifica automaticamente o tipo do dado atribuído.
- Texto (str),
- Inteiro (int),
- Decimal (float),
- Booleano (bool).
- Comentários #
- Docstrings `"""`





É hora do quiz!

Prepare-se para
algumas
perguntas.



<https://forms.gle/reAgzAsDcz7iE5j477>

Interatividade, Operadores

 Saída: `print()`

 Entrada: `input()`

Relacionais:

$>$	$\geq$	$==$
$<$	$\leq$	

Aritiméticos:

$+$	$*$	$/$	$\%$
$-$	$**$	$//$	



Strings e formatação

Conversão de Tipos: `bool()`; `int()`; `float()`; `str()`;

Concatenação (+)

Repetição (*)

f strings `f "Texto {variável}."`

Aspas `" ' ' "`

Escape \

substrings `[:]`

`f"{valor:.2f}"` e `f"{valor:.,}"`,

P - Y - T - H - O - N



Métodos



- **upper()** e **lower()**
- **capitalize()** e **title()**
- **len()**
- **strip()**
- **ord()** e **chr()**
- **texto.replace("texto_antigo", "texto_novo")**
- **texto.split() = []**
- **texto.find("texto_procurado")**
- **texto.count("texto")**

Mais um Quiz!

Prepare-se para
as perguntas.



<https://forms.gle/THjj18cnjN84gaVVA>

Operadores lógicos e Condições

- and, or , not

if condicao01 == condicao 02:

acao

elif condicao02 != condicao 03:

acao

else:

acao

Se... eu vejo um
carteiro, vou direto
nas tornozelos dele.



Ótimo
exemplo!



Operador ternário (Expressão Condicional):

valor_se_verdadeiro **if** condicao **else** valor_se_falso

Operadores lógicos e Condições

match entrada:

case “condicao01” | “condicao02”:

acao

case _:

acao

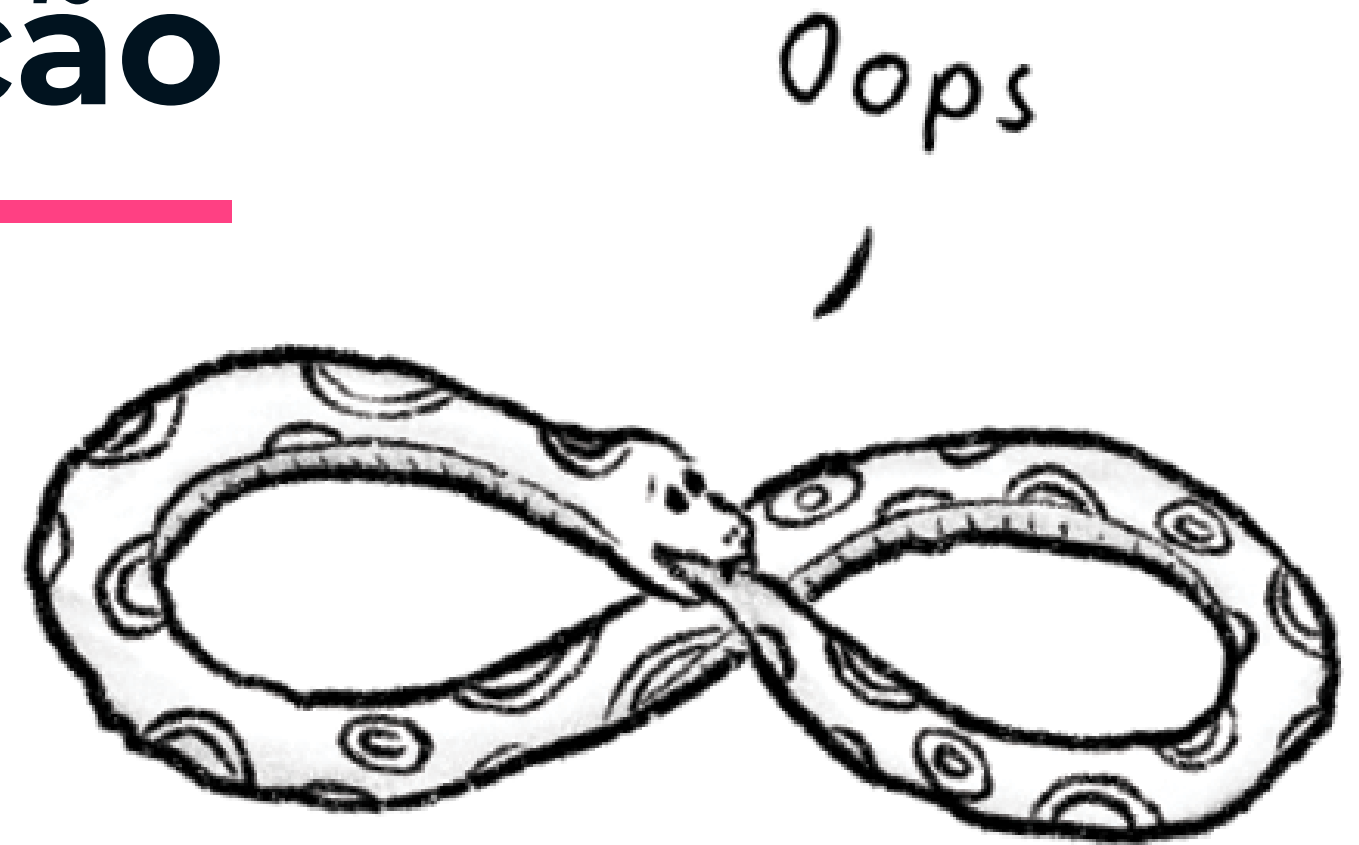
Estruturas de repetição

while condicao:
acao

for variavel **in** sequencia:
acao

+=

-=



range(numero) / range(inicio, fim, intervalo)

Ignorar ou parar

```
for variavel in sequencia:  
    if condicao01:  
        continue  
acao
```

```
while True  
    if condicao02:  
        break  
acao
```



Funções

```
def nome_da_função (parametro):
```

```
    """
```

```
    doc
```

```
    """
```

```
    acao
```

```
nome_da_função (argumento)
```

Valor padrão:

```
def nome_da_função (parametro01, parametro02 =
```

```
    “valor padrão”):
```

```
    acao
```



Funções



argumentos nomeados:

```
def nome_da_função (parametro01, parametro02):  
    acao
```

```
nome_da_função (parametro02 = "valor02", parametro01 =  
"valor01")
```

args** ou *kwargs** - múltiplos argumentos

```
def nome_da_função (*parametro01, parametros02):  
    acao
```

```
nome_da_funcao (var01, var02, var03, chave01=valor01)
```

Funções

Retorno em funções

```
def nome_da_função (parametro01):  
    acao  
    return algo
```

Escopo das variáveis

locais x globais

Globais = Leitura livre, alteração mediante **global**

Locais = Leitura somente dentro da função, ou em função mais interna, alteração mediante **nonlocal**



Quiz?



<https://forms.gle/Q1kcDrKqLfNNM96FA>

Estruturas de dados: Lista

lista[item01, item02]

Métodos e operações:

lista[1:4]

.append(elemento) # adiciona ao final;

.insert(*índice*, elemento) # adiciona na posição *índice*;

.remove(elemento) # remove a primeira ocorrência;

.pop() # remove o último elemento;

.pop(*índice*) # remove o elemento na posição *índice*;

.reverse # inverte a ordem da lista;

.sort # ordem alfabética;

len(lista)

item **in** lista (True ou False)

.clear() # limpa a lista

[] *Listas*

- Mutáveis
- e ótimas para flexibilidade

Estruturas de dados: Tuplas

tupla(item01, item02)

Desempacotamento

item01, item02 = tupla

.count(elemento) # quantidade de vezes que o elemento aparece;

.index(elemento) # índice da primeira ocorrência;

len(tupla)

() Tuplas

- (Feimosa)
- Imutável
- Quando você precisa que as coisas permaneçam iguais

Estruturas de dados: Dicionários

dicionario{chave:valor}

Métodos e operações:

dicionario[chave] # acessa o valor da chave;

.get(chave) # acessa o valor da chave;

dicionario[chave] = novo_valor # altera o valor da chave;

dicionario[nova_chave] = valor # adiciona uma chave;

.pop(chave) # remove a chave;

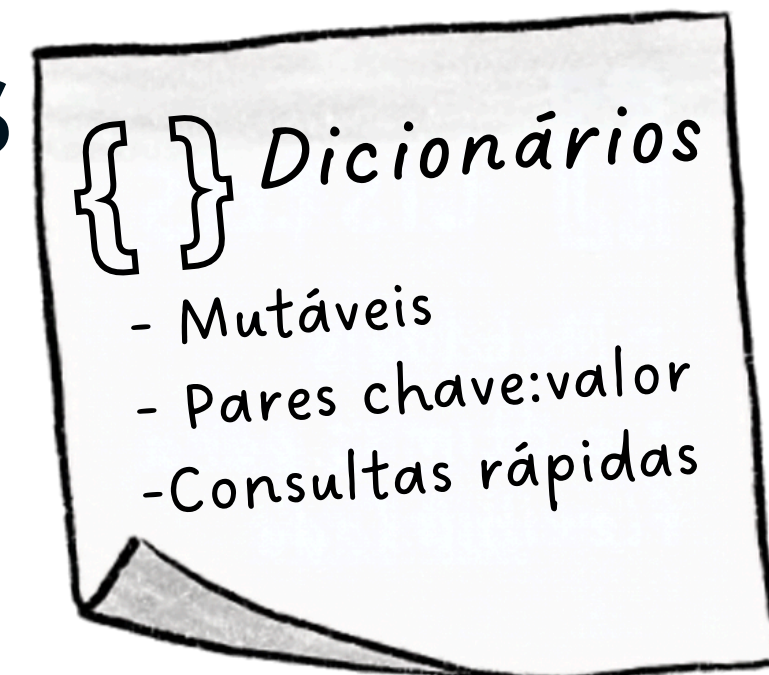
del dicionario[chave] # remove a chave;

.keys() # Retorna as chaves do dicionário

.values() # Retorna os valores

.items() # Retorna pares (chave, valor)

list(elementos) # transforma o valor numa lista



Estruturas de dados: Conjuntos

set(nome_de_uma_lista)

set Conjuntos

(())

- Elementos únicos
- Mutável
- Útil para operações matemáticas

.add(elemento) # adiciona um elemento

.discard(elemento) # remove um elemento

item **in** tupla #(True ou False)

conjunto01.**union**(conjunto02) #união

conjunto01.**intersection**(conjunto02) # intersecção

conjunto01.**difference**(conjunto02) # diferença

Estou
acertando
tudo!



Isso pode ser um
bug...

<https://forms.gle/MBvqRyyx9X59tSMT7>

Bugs

Syntax Error - Regras gramaticais quebradas.

Runtime Error - Erro durante a execução do código.

Erro Lógico (Logical Error) - Resultado incorreto, não esperado.

Técnicas:

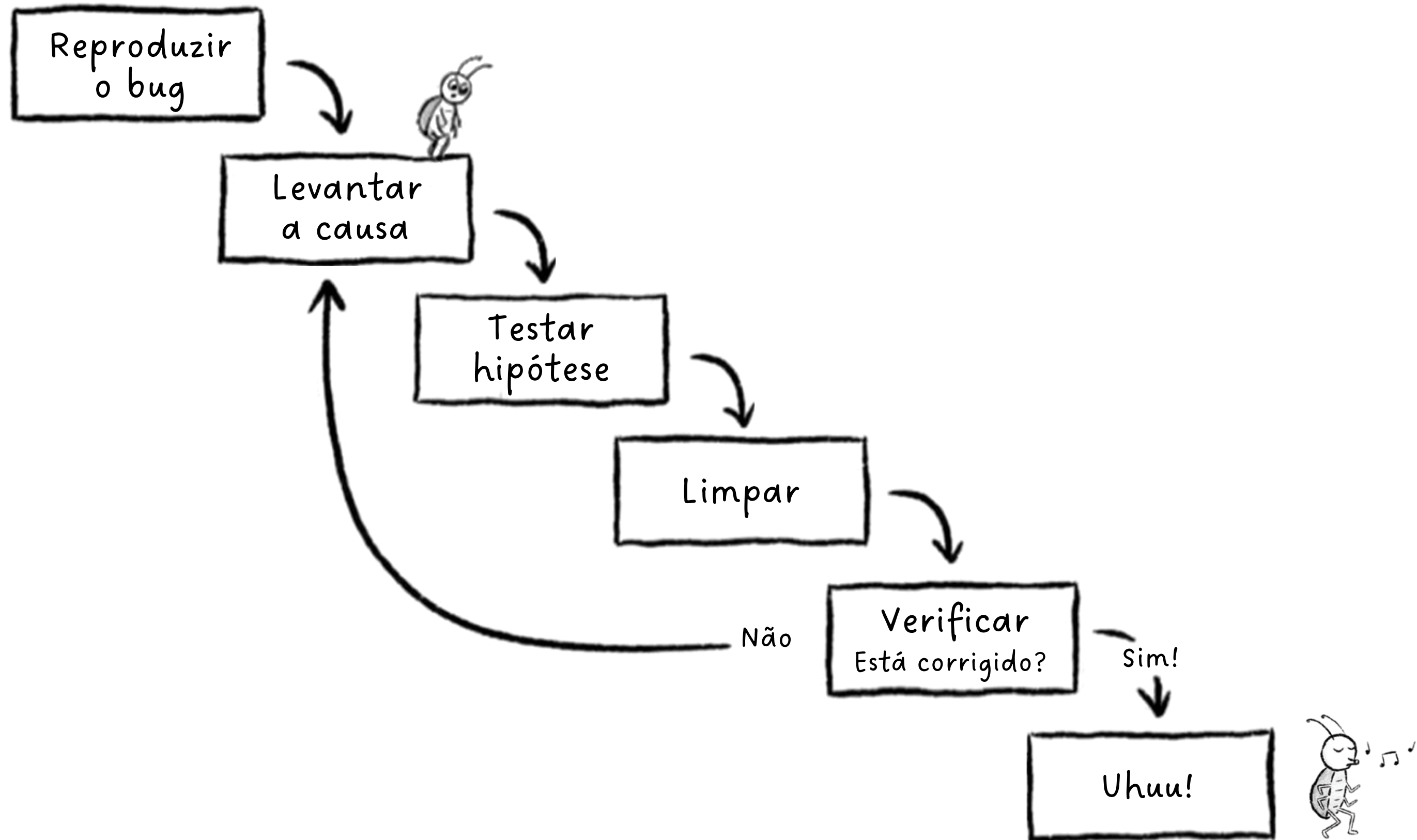
traceback
print()

depurador da IDE (breakpoint, call stack, variables)

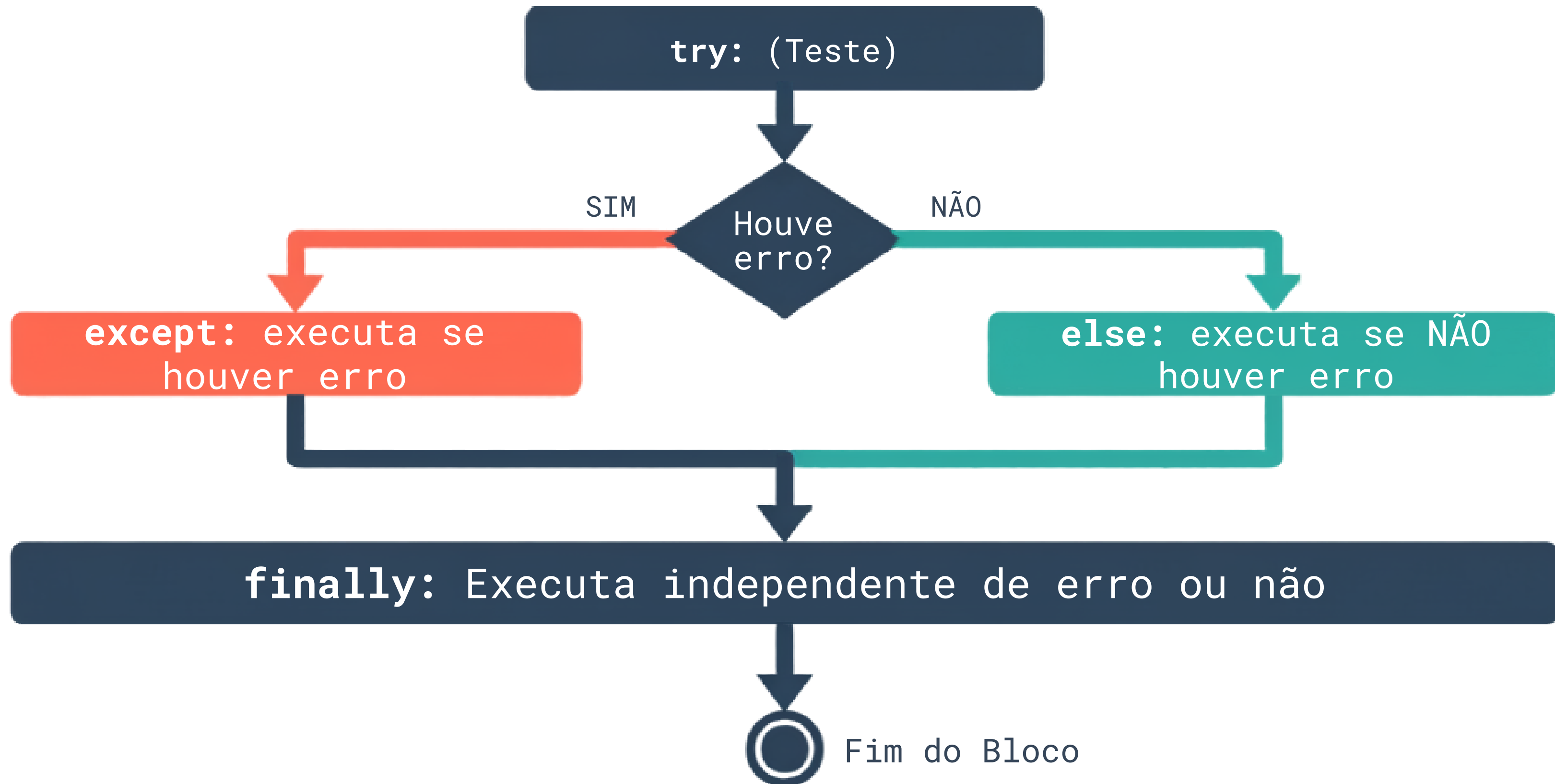
módulo pdb - breakpoint()



Fluxo de Trabalho de Depuração



Tratamento de Erros





<https://forms.gle/TRpk7URyDPw1bjmV8>

Próximos Passos

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

**Prova.
Bons Estudos!**



**exit() ou
dúvidas?**

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

OBRIGADA!

“Errors should never pass silently.
Unless explicitly silenced.”

The Zen of Python, by Tim Peters



Referências: PUTTEN, Maaïke van; PUTTEN, Imke van. Python Illustrated: Not another boring Python book, learn programming the fun way. Birmingham: Packt Publishing, 2026. [[1](#), [2](#), [3](#)]

SANTA CRUZ

CENTRO UNIVERSITÁRIO

Para mais
informações
acesso
o **QR CODE**
e saiba mais



 (41)3052-4900

 @unisantacruz

 /UniSantaCruzCtba

 unisantacruztem.com.br